Reviewer Pack — Matroid Representation Complexity of Monotone DNF for k-CLIQ…

Entry 499c5d46d38e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 21:44:43 UTC

Matroid Representation Complexity of Monotone DNF for k-CLIQUE

Entry ID: `499c5d46d38e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 21:44:43 UTC

1. Conjecture

Field A (mathematical branch): Matroid Representability over Finite Fields **Field B** (complexity object): Monotone DNF Formulas

Statement:

> For a monotone DNF formula Φ over n variables, define μ(Φ) as the minimal size of a finite field representation of its term lattice. Then, μ(Φ) ≤ O(log n) for any Φ with O(poly(n)) terms, and μ(k-CLIQUE) ≥ Ω(n^{1-1/k}) for k ≥ 3.

Rationale (proposer's reasoning):

> Matroid representability over finite fields captures combinatorial structure of DNF terms. The conjecture links this to k-CLIQUE's inherent complexity, suggesting that k-CLIQUE's term lattice requires exponentially larger representations than general DNFs.

Taxonomy category: `DISPERSION_DISCREPANCY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `eecd80a3a1a241cd`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_dnf(n, num_terms):
        variables = list(range(n))
        terms = []
        for _ in range(num_terms):
            term = set(random.sample(variables, random.randint(1, n)))
            terms.append(term)
        return terms

    def matroid_representation_size(terms):
        # Simple heuristic to estimate the size of a matroid representation
        # This is a placeholder and should be replaced with an actual algorithm
        return len(terms) * 2

    n = random.randint(5, 40)
    num_terms = random.randint(10, min(n**2, 100))
    dnf = generate_dnf(n, num_terms)

    mu_phi = matroid_representation_size(dnf)

    if len(dnf) > 100:
        return {
            "metric_name": "matroid_representation_size",
            "metric_value": mu_phi,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    k = 3
    lower_bound = n**(1 - 1/k)

    return {
        "metric_name": "matroid_representation_size",
        "metric_value": mu_phi,
        "instances_tested": 1,
```

```
        "conjecture_holds": mu_phi <= 2 * math.log(n) and mu_phi >= lower_bound,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ize', 'metric_value': 58, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'matroid_representation_size', 'metric_value': 28, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'matroid_representation_size', 'metric_value': 112, 'instances_tested': 1, 'c
TRIAL: {'metric_name': 'matroid_representation_size', 'metric_value': 106, 'instances_tested': 1, 'c
TRIAL: {'metric_name': 'matroid_representation_size', 'metric_value': 36, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'matroid_representation_size', 'metric_value': 62, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'matroid_representation_size', 'metric_value': 146, 'instances_tested': 1, 'c
TRIAL: {'metric_name': 'matroid_representation_size', 'metric_value': 96, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'matroid_representation_size', 'metric_value': 130, 'instances_tested': 1, 'c
TRIAL: {'metric_name': 'matroid_representation_size', 'metric_value': 154, 'instances_tested': 1, 'c

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_62da5526.py", line 77, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_62da5526.py", line 77, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                                ~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

Test crashed before producing valid results; no counterexample found but conjecture_holds is False for multiple trials | next: Implement error handling for 'seed' field and re-run tests with sufficient sample size

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	66814
2	propose	ollama_remote	qwen3:8b	0	0	108042
3	preregistration	ollama_remote	qwen3:8b	0	0	24131
4	novelty	ollama_remote	qwen3:8b	0	0	20702
5	novelty	ollama_remote	qwen3:8b	0	0	15887
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11652
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7309
8	judge	ollama_remote	qwen3:8b	0	0	15263

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 269799 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/499c5d46d38e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/499c5d46d38e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/499c5d46d38e.tar.gz` (if generated)